

IncidentIQ Next Steps Recommendation

Generated May 10, 2026 from the IncidentIQ project review.

Prepared from the current IncidentIQ repository review.

Recommended Direction

Treat the current project as an MVP that already works, then move it toward a clean production-style architecture without breaking the demo. Preserve the working `src/pipeline.py` behavior, extract modules gradually, add tests around each extraction, then introduce heavier LangGraph and agent abstractions after the deterministic core is stable.

1. Stabilize The Repo State

1. Review the current git status and decide which moved files are intentional:
 - `notebooks/` moved to `docs/notebooks/`
 - `root knowledge_base/` moved to `src/rag/knowledge_base/`
 - local docs updated accordingly
2. Make sure moved files are represented as renames rather than delete/add noise if preserving git history matters.
3. Keep `.env` out of git.
4. Add or update `.env.example` with safe values:

```
INCIDENTIQ_WORK_ROOT=/tmp/incidentiq_work
workDir=/absolute/path/to/IncidentIQ/src/rag
ENABLE_RAG_MODE=true
ENABLE_EVAL_MODE=true
ENABLE_PREVIEW_MODE=true
ENABLE_REAL_INTEGRATIONS=false
INCIDENTIQ_RAG_TOP_K=5
PORT=8765
GRADIO_SERVER_NAME=127.0.0.1
JIRA_BASE_URL=https://jira.example.com
JIRA_PROJECT_KEY=OPS
SLACK_CHANNEL=#devops-incidents
```

2. Add A Real Test Harness

1. Create a `tests/` directory.
2. Add focused tests for the deterministic core:
 - ingesting `.zip`, `.jsonl`, `.log`, `.txt`
 - skipping `ground_truth_eval_only`
 - parsing malformed lines without crashing
 - extracting known signals

- clustering evidence
 - loading RAG docs from `workDir`
3. Add one integration test using `datasets/incidentiq_small_test_sample_easy.zip`.
 4. Keep `scripts/smoke_test.py` for demos and add a pytest equivalent for CI.

3. Extract `src/pipeline.py` Gradually

Do not rewrite the pipeline all at once. Extract stable chunks while keeping public behavior identical.

Recommended order:

1. `src/config/settings.py`
 - Move env parsing there.
 - Include `workDir`, `INCIDENTIQ_WORK_ROOT`, feature flags, Jira config, Slack config, and RAG top-k.
 - Keep defaults centralized.
2. `src/ingest/`
 - Extract upload and file discovery.
 - Extract safe ZIP handling.
 - Extract log parsing.
 - Preserve malformed-line behavior.
3. `src/signals/`
 - Move signal rules and severity heuristics.
 - Keep rule definitions deterministic and testable.
4. `src/clustering/`
 - Move evidence grouping.
 - Keep current grouping behavior first.
 - Add service, time window, trace ID, and repeated-signature improvements later.
5. `src/reporting/`
 - Extract Markdown, JSON, CSV, and export bundle generation.
 - Keep output filenames and schemas stable.

4. Formalize Models

If `src/pipeline.py` owns many Pydantic models, move them into `src/models/`.

Recommended minimum model files:

- `src/models/enums.py`
- `src/models/log_event.py`

- `src/models/signal.py`
- `src/models/cluster.py`
- `src/models/incident.py`
- `src/models/actions.py`
- `src/models/reports.py`

Rule of thumb: anything crossing module boundaries should be a Pydantic model, not a loose dictionary.

5. Improve RAG Cleanly

1. Keep `workDir` as the single RAG base directory if that is the required convention.
2. Add tests confirming:
 - default RAG root is `src/rag`
 - `workDir=/path/to/src/rag` derives `/path/to/src/rag/knowledge_base`
 - `workDir=/path/to/src/rag` derives `/path/to/src/rag/faiss_index`
 - `_read_docs()` returns documents with source paths
3. Normalize RAG chunk output to match remediation needs:
 - `diagnostics`
 - `remediation`
 - `validation`
 - `safety_notes`
 - `source`
 - `title`
 - `score`
4. Keep lexical RAG as the default fallback. Add FAISS only when dependencies are available.

6. Add LangGraph After The Core Is Modular

Do not force LangGraph in too early. First create module boundaries, then wrap them.

Recommended graph nodes:

1. `ingest_logs`
2. `normalize_events`
3. `extract_signals`
4. `cluster_evidence`
5. `retrieve_runbooks`
6. `classify_incident`
7. `build_timeline`

8. `generate_rca`
9. `recommend_remediation`
10. `critic_review`
11. `build_notification_preview`
12. `build_ticket_preview`
13. `synthesize_cookbook`
14. `build_reports`
15. `optionally_score_evals`

Start with deterministic node functions. Add LLM-backed agents after the graph runs without them.

7. Separate Runtime From Eval

1. Runtime paths must never read `ground_truth_eval_only`.
2. Evals should only run after inference produces output.
3. Add tests that prove runtime skips ground truth.
4. Consider writing inference outputs first, then passing the output path into eval scoring.

8. Improve The Gradio App

1. Add clearer UI states for:
 - upload received
 - parsing
 - signal extraction
 - clustering
 - RAG retrieval
 - report generation
2. Surface top incidents in a compact table.
3. Show RAG sources per incident.
4. Keep Jira and Slack as preview-only.
5. Add a visible "real integrations disabled" indicator when `ENABLE_REAL_INTEGRATIONS=false`.

9. Documentation Cleanup

1. Keep `README.md` focused on usage and demo value.
2. Keep `ReadMe_Local_Deployment.md` focused on local setup.
3. Keep `src/src_README.md` focused on architecture.

4. Update architecture docs so they describe the current system rather than old gaps that are already fixed.
5. Add a short Known Limitations section:
 - LLM mode optional or not fully wired
 - FAISS optional
 - real Slack/Jira disabled
 - notebooks are historical/prototype assets

10. Suggested Immediate Sprint

Recommended 1-2 day sequence:

1. Create `.env.example`.
2. Add pytest setup.
3. Add tests for RAG path derivation and `_read_docs()`.
4. Add tests for smoke sample pipeline behavior.
5. Extract settings into `src/config/settings.py`.
6. Update `src/rag/rag_retriever.py` to consume settings cleanly.
7. Extract ingest logic from `src/pipeline.py`.
8. Confirm Gradio and smoke test still pass.
9. Commit the path migration and env cleanup separately from code extraction.

Best Next Move

Lock in tests before refactoring `src/pipeline.py`. That gives the project a safety net while turning the working MVP into a maintainable architecture.